Work Notes: 31 Oct 2026 (Dialer report rewrite, multi-day CDR, Uno API, trading days)

Oct 31, 2026 · @tanvik

Overview

The dialer report was rebuilt as a fast, checked script that now covers any number of days, and testing it exposed three bugs in the old notebook. The longest problem of the day, a report showing only the latest date, turned out to be the script reading the wrong folder.

#	Task	Tool	Key idea to learn
1	`FILTER` with `'BILL'`	Excel	Double quotes for text, single quotes for sheet names
2	Organised Stringee CDR script	pandas	`pathlib`, fast Excel engines, `# %%` notebook cells
3	Reading types correctly	pandas	Phone numbers as text at read time; durations converted after
4	Dialer report rewrite	pandas	Settings in one place; test against the old version before trusting it
5	Vectorised call gaps	pandas	`groupby().shift()` instead of a row loop
6	Multi-day reports	pandas	Read every file; per-day keys; checks that fail loudly
7	Only the latest date appeared	pandas	Check which files are read before debugging code
8	Small settings changes	pandas	Keep `CAN_NOT_MAKE_CALL`; newest team list in a folder
9	Uno API	Python	Environment variables hold secrets by name
10	Client trading days	pandas	`nunique` of dates, not a count of rows

The end-of-day review comes first, then one section per task, then the error log, cheat sheet and open items.

End-of-day review (day 3 of 7)

You asked 26 questions today (21 pandas, 4 Python and API, 1 Excel), and the dialer report is now fast, tested and multi-day. The biggest time cost was 11 rounds spent on "only the latest date" before the file list showed the script was reading an old folder.

Solved today

- A clean Stringee CDR script in notebook-cell format, with the right types for times, durations and phone numbers.

- The dialer report rebuilt, tested cell-for-cell against the old notebook, with three old bugs fixed.
- Multi-day support: every file read, hourly report per CRE per day, overlapping exports removed.
- The folder mix-up found and fixed; `CAN_NOT_MAKE_CALL` kept; the team list read from a folder.
- The Uno token stored as a Windows environment variable.
- Client activity counted in trading days instead of bills.

Not confirmed yet

- The dialer report's first full run on 1 to 30 Oct (check `Calls per day`).
- The trading-days report rerun with a maximum of 31 or less.
- The Uno API endpoint and parameters, and a regenerated token.
- Which Uno "connected" rule to use (`answered_flag` or `both_answered`).

Mistakes that came back

Pattern	Earlier	Today
Debugging the code before checking the inputs	Ledger "stopped at the 27th" (30 Oct)	"Only the 28th": the script read `Documents\Dialer\st_daily`, not `Videos\...\CDR\Stringee`
Trusting output from an older run	Cells run out of order (24 Oct); `df` overwritten (30 Oct)	The trading-days file still showed bill counts
Mixing up a name and a value	—	Token used as the variable name; `'BILL'` in sheet-name quotes
Duplicate definitions	—	Your settings cell defines `read_xlsx`, `LEVERAGE_PER_CALL` and others twice; the last one silently wins

On the folder problem, part of the delay was mine: I chased a header-name theory before asking which files were being read. The script printed `Stringee files: ['20261029_052117.xlsx', ...]` on every run, and comparing that list with the file you expected would have found it in one step.

Gaps to study

- **Inputs first:** print the file list and each file's date range before reading any result.
- **Notebook state:** Restart and Run All before trusting a final number.
- **Grouped calculations:** `groupby().shift()` for previous-row logic; `nunique` versus `count`.
- **APIs:** authentication, environment variables, paging by date.

Study programme check

No study chats since 29 Oct, so PG40 Day 1 is still at 0 of 40 in the chats I found. Yesterday's focus (exercises 1 to 10) didn't happen, so tomorrow's is smaller.

Focus for tomorrow

PG40 Day 1, exercises 1 to 5, answers sent in chat.

Your chats now cover more than 7 days, so the 7-day study plan can be drafted any time you ask.

1. Excel: text in double quotes, sheet names in single quotes

`'BILL'` in single quotes made Excel reject the formula. In Excel, single quotes only wrap sheet names; text values always take double quotes.

```
=UNIQUE(FILTER('31 oct 2026'!C:C, '31 oct 2026'!E:E="BILL", "No matches"))
```

Quote	Used for	Example
`'...'`	A sheet name with spaces or symbols	`'31 oct 2026'!C:C`
`"..."`	A text value	`"BILL"`, `"No matches"`

- The comparison ignores capitals, so `Bill` matches, but `"BILL "` with a trailing space doesn't. Check with `=COUNTIF(E:E, "*BILL*")`.
- `"No matches"` replaces the `#CALC!` error when nothing matches.
- Whole-column ranges are slow; fixed ranges of equal size (`C2:C100000`, `E2:E100000`) are faster.

Practice

- [ ] Return unique clients with BILL or DEPOSIT using `(E:E="BILL") + (E:E="DEPOSIT")`.

2. Stringee daily CDR: organised, faster, in notebook cells

The combine-and-save script now keeps all settings at the top, reads with full paths instead of `os.chdir`, and uses faster Excel engines when installed. It lives in `stringee_daily_cdr.py` as `# %%` cells, ready to convert into a notebook.

The core of it

```
from pathlib import Path
import pandas as pd

INPUT_DIR = Path(r"C:\Users\Administrator\Videos\Office\Dialer\CDR\Stringee")
OUTPUT_DIR = Path(r"C:\Users\Administrator\Videos\Office\Dialer\Processed
CDR\Stringee")

def read_xlsx(path):
    try:
        return pd.read_excel(path, engine="calamine")      # much faster, if
installed
    except ImportError:
        return pd.read_excel(path)

files = sorted(f for f in INPUT_DIR.glob("*.xlsx") if not
f.name.startswith("~$"))
if len(files) != 2:
    raise ValueError(f"Expected exactly 2 .xlsx files, found {len(files)}:
{[f.name for f in files]}")

frames = [read_xlsx(f) for f in files]
concat_df = pd.concat(frames, ignore_index=True)
concat_df["Start time"] = pd.to_datetime(concat_df["Start time"],
format="ISO8601")
print("Exact duplicate rows:", concat_df.duplicated().sum())
```

What changed and why

Before	After	Why
`if len(xlsx_files) > 2`	`!= 2`, listing the files found	With 0 or 1 file, the old check passed and then failed on `xlsx_files[1]`
`os.chdir(path)`	`Path` objects with full paths	Changing the working folder can break later cells
Lock files read as data	`~$` files skipped	Excel creates them while a workbook is open
`format='%Y-%m-%d %H:%M:%S.%f'`	`format="ISO8601"`	Accepts times with and without fractions of a second
Default Excel engine	`calamine` to read, `xlsxwriter` to write	Both much faster; the helpers fall back if not installed
Unused imports	Removed	`datetime`, `timedelta`, `numpy` weren't used

Install the fast engines once with `pip install python-calamine xlsxwriter`.

Notebook-cell format

- `# %%` starts a code cell; `# %% [markdown]` starts a text cell.
- In VS Code each cell shows **Run Cell**. To make an `.ipynb`: Command Palette, then **Jupyter: Export Current Python File as Jupyter Notebook**.
- From a terminal: `pip install jupytext`, then `jupytext --to notebook stringee_daily_cdr.py`.

Practice

- [ ] Time one run with and without `python-calamine` installed, using `%%time` at the top of the cell.

3. Reading types correctly: dates, durations and phone numbers

Only ID and phone columns must be set while reading; dates and durations are converted right after. Testing showed why: a blank cell turns a phone column into floats like `9.19225131053e+11`, and once that happens a later conversion gives `'919225131053.0'`.

Column	How pandas reads it	What to do
`Start time`, `End time`	Real date-times if Excel stored them as dates	`pd.to_datetime` after reading; it leaves real dates as they are
`Answer duration` and other durations	Excel time-of-day values or text; neither can be summed	`pd.to_timedelta` after reading
`Customer number`, `Hotline`, `ID`	Numbers, or floats if any cell is blank	`dtype=str` at read time

```
TEXT_COLS = {"ID": str, "Customer number": str, "Hotline": str, "Company ID":
str}
df = pd.read_excel(path, dtype=TEXT_COLS)

def to_duration(s):
    """'00:05:40' text or Excel time values -> timedelta; blanks -> NaT."""
    return pd.to_timedelta(s.astype(str).str.strip(), errors="coerce")

for col in ["Queue duration", "Answer duration", "Hold duration"]:
    df[col] = to_duration(df[col])
```

Saving durations back to Excel

pandas writes a timedelta as a fraction of a day, so `00:05:40` appears as `0.003935`. Give those cells the `[h]:mm:ss` format and they display correctly and still add up with `SUM`:

```
out = df.copy()
for c in DURATION_COLS:
    out[c] = out[c].dt.total_seconds() / 86_400          # Excel stores time
as a fraction of a day

with pd.ExcelWriter(path, engine="xlsxwriter") as writer:
    out.to_excel(writer, index=False, sheet_name="CDR")
    fmt = writer.book.add_format({"num_format": "[h]:mm:ss"})
    for c in DURATION_COLS:
        i = out.columns.get_loc(c)
        writer.sheets["CDR"].set_column(i, i, 14, fmt)
```

The numbers had to be converted first: when pandas writes timedeltas itself, it gives each cell its own `0` format, which overrides the column format.

Using durations in pandas

```
df["Answer duration"].dt.total_seconds()                # seconds as numbers
df["Answer duration"].sum()                             # total talk time
df[df["Answer duration"] > pd.Timedelta(seconds=30)]     # calls over 30
seconds
```

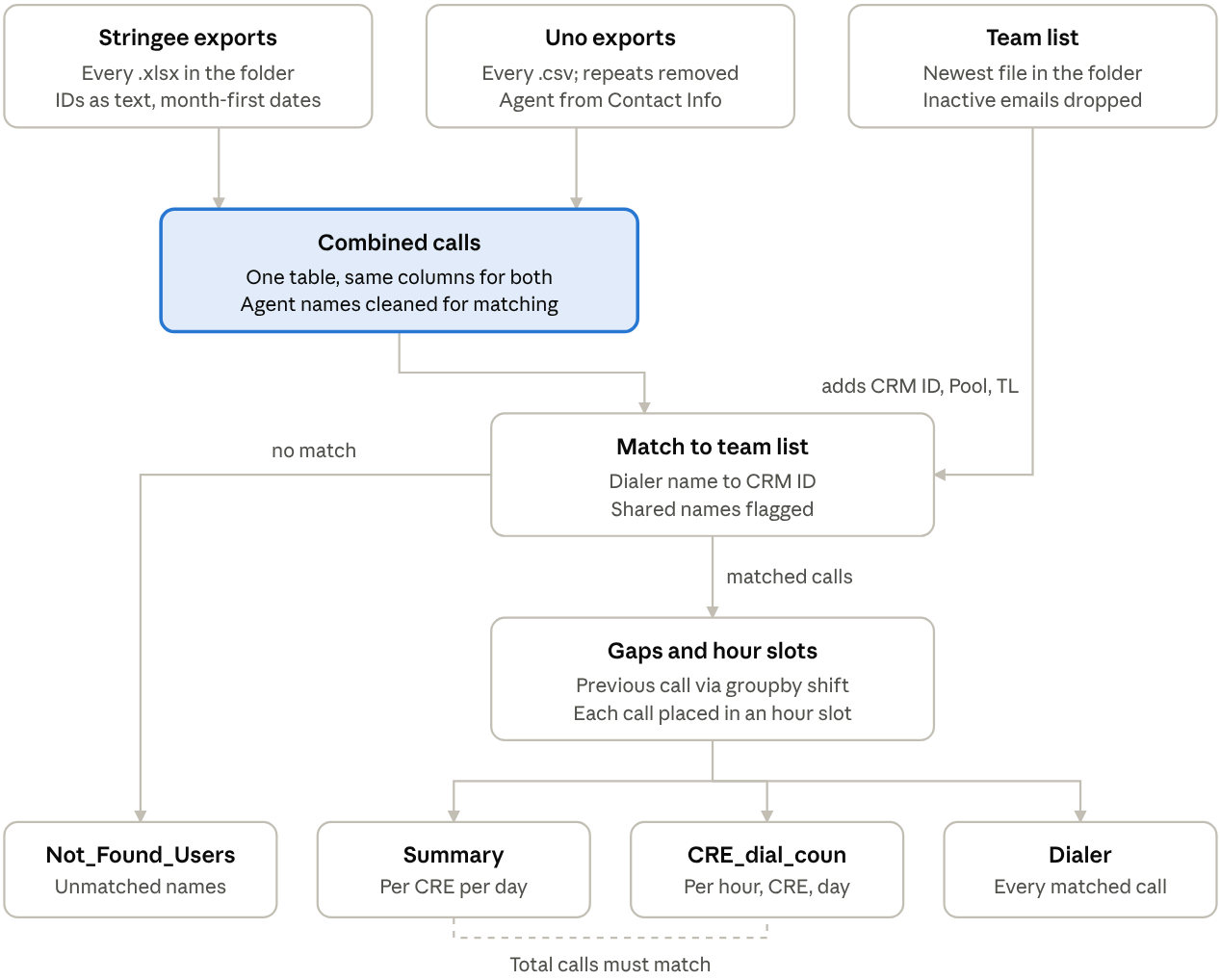
Practice

- [ ] Read a file with and without `dtype=str` on `Customer number` and compare the values in a row with a blank.

4. Dialer report rewrite (`dialer_report.py`)

The new script produces the same four reports as the Stringee + Uno notebook, matched it cell for cell on Stringee data, and runs a test day of 8,162 calls in 2.1 s instead of 7.9 s (0.6 s without saving the raw Excel copies).

Both dialers become one call table before any report is built



dialer_report.py · 3 inputs, 4 reports

Stringee and Uno are reshaped into one table first, so matching, gaps and every report run once for both dialers.

How it was checked

A faithful copy of the old notebook and the new script ran on the same test days, including overlapping exports, IVR calls with no agent, a duplicated team-list row and an inactive user. With matching settings, the Summary, hourly report and call-level file had 0 differing cells for Stringee CREs. Each multi-day run was also compared against running every day on its own.

Three bugs found in the old notebook

Bug	Cause	Effect in the test
Every Uno call lost its start time	The date was glued to Uno's full date-time: `'2026-10-30 2026-10-30 09:12:00'` fails to parse	All 176 Uno calls in the `20+` slot, no gaps, blank first/last call
Every Uno call duration became 0	A later step turned anything that wasn't text into `Timedelta(0)`; Uno durations were already timedeltas	`Total_Duration` 00:00:00 and Login Hours 01:00:00 instead of 04:35:19 and 08:32:22
Hourly talk time sometimes 1 second short	`int(mins * 60)` after converting to minutes; `100/60*60` gives `99.99999…`	9 of 767 cells off, e.g. 00:08:11 instead of 00:08:12

Settings worth knowing

Setting	Default	Meaning
`EXCLUDE_STRINGEE_END_CODES`	`[]` (changed today)	End codes to drop; `["CAN_NOT_MAKE_CALL"]` drops failed dials
`DEDUPE_NUMBER_AND_TIME`	`True`	Removes repeated Number + Call Start Time, as your first version did
`UNO_CONNECTED_RULE`	`"answered_flag"`	`"both_answered"` counts only Status = Both Answered; in the test, 80 vs 40 connected for one agent
`SAVE_PROCESSED_RAW`	`True`	Also writes cleaned copies of the raw files

Each run prints a Status by Answered table for Uno. In your sample, `Answered` was TRUE on `From Ans. - To Unans.` calls, where the customer never picked up.

Rules kept as they were, but worth reviewing

- Pool 1's warning band is 22 to 24 minutes, so 24 to 25 minutes of talk counts as "Not Complete".
- Other pools only reach "Warning" at exactly 22.
- `Total Unique Calls` adds the hourly unique counts, so a number dialled in two hours counts twice.

Practice

- [ ] Run once with each `UNO_CONNECTED_RULE` and compare `Total_Connected_Calls` for two Uno agents.

5. Vectorised call gaps: `groupby().shift()` instead of a loop

`shift()` moves a column down one row inside each group, so every call can see the previous call's start and duration at once. That replaces the old row-by-row `for` loop with `.loc`, which was the slowest part of the notebook and got much slower on bigger days.

```
dialers = dialers.sort_values(["Date", "CRM ID", "Call Start Time"],
kind="mergesort").reset_index(drop=True)

by_cre_day = dialers.groupby(["Date", "CRM ID"], sort=False)
prev_start = by_cre_day["Call Start Time"].shift()
prev_end = prev_start + by_cre_day["Total Call Duration"].shift()
gap = (dialers["Call Start Time"] - prev_end).clip(lower=pd.Timedelta(0))

start_tod = dialers["Call Start Time"] - dialers["Call Start
Time"].dt.normalize()
prev_tod = prev_start - prev_start.dt.normalize()
in_hours = start_tod.between(WORK_START, WORK_END) &
prev_tod.between(WORK_START, WORK_END)
flagged = in_hours & (gap.dt.total_seconds() >
GAP_FLAG_SECONDS).fillna(False)
```

One CRE's morning, step by step

Start	Duration	Previous start (shift)	Previous end	Gap	Call Gap
09:40:00	00:02:00	(none: first call)	(none)	0	No

Start	Duration	Previous start (shift)	Previous end	Gap	Call Gap
09:42:30	00:00:30	09:40:00	09:42:00	00:00:30	No
09:46:00	00:05:00	09:42:30	09:43:00	00:03:00	Yes
09:50:00	00:01:00	09:46:00	09:51:00	0 (overlap, clipped)	No

How each part works

- **Sort first.** `shift()` takes the row above, so rows must be in time order within each CRE and day. `kind="mergesort"` keeps ties in a fixed order, so results don't change between runs.
- **`groupby(...).shift()`** restarts at each new CRE or day, so the first call never looks at someone else's last call.
- **`.clip(lower=pd.Timedelta(0))`** turns overlaps (a call starting before the previous one ended) into 0, as the loop did.
- **Time of day** is the start minus its own midnight, which gives a timedelta to compare with 09:30 and 18:30.
- **Sums without lambdas.** The summary pre-computes columns such as `talk_s.where(conn, 0)`, then uses plain `"sum"`. The old per-group `lambda`s with `Dialers.loc[x.index]` were the second slowest part.

Practice

- [ ] Add a column with the gap *before* each call in seconds, then find each CRE's longest gap of the day with `groupby(...).max()`.

6. Multi-day reports: every file, per-day keys, loud checks

The script now reads every file in both folders and builds the hourly report per CRE per day. On a 3-day test (six Stringee files, three Uno CSVs plus one overlapping export), each day matched a single-day run exactly and 50 overlapping Uno rows were removed.

Changes, cell by cell

Cell	Change	Why
Stringee	`if not st_files: raise ...` instead of `!= 2`	Any number of exports

Cell	Change	Why
Uno	Read every CSV with `pd.concat`; `drop_duplicates(subset="Call Id")`	Overlapping exports counted once
Summary	Sort by `["Date", "TL", "CRM ID"]`	Readable across several days
Hourly	`keys` starts with `"Date"`; merges on `["CRM ID", "Date"]`	One row per CRE per day, no repeated rows
Hourly	Calls pivot counts `"Call Status"`, not `"Date"`	A column can't be both a key and the value counted
Export	`date_range_name(prefix, dialers["Date"], ".csv")`	`Summary_2026-10-28_to_2026-10-30.csv`

The error: `Grouper for 'Date' not 1-dimensional`

Adding `"Date"` to the pivot keys broke the Calls pivot, which counted `values="Date"`. pandas couldn't use one column as both a group key and the value being counted. Counting `"Call Status"` instead gives the same number, because it's never blank.

Checks that fail loudly

```
check = hourly[on + ["Total Calls"]].merge(summary[on +
["Total_Dialed_Calls"]], on=on, how="left")
mismatch = check[check["Total Calls"] != check["Total_Dialed_Calls"]]
print("Hourly totals match the summary:", mismatch.empty)
print("Calls per day:", dialers.groupby(dialers["Date"].dt.strftime("%Y-%m-
%d")).size().to_dict())
```

Two reports built from the same calls must agree. When they don't, a join or filter has gone wrong, and the check says so before anyone reads the file.

Export names with the date range

```
reports = {"Summary": summary, "Dialer": dialer_out, "Not_Found_Users":
not_found, "CRE_dial_coun": hourly}
for prefix, df in reports.items():
    name = date_range_name(prefix, dialers["Date"], ".csv")
    df.to_csv(REPORT_DIR / name, index=False)
```

Practice

- [ ] Add a check that every date between the first and last date appears in `Calls per day`, and print any missing days.

7. Debugging "only the latest date": it was the folder

The report showed only 28 Oct because `STRINGEE_DIR` pointed to `Documents\Dialer\st_daily`, which held just that day's two exports. The full export from 1 Oct was in `Videos\Office\Dialer\CDR\Stringee`.

How it was found

1. The summary code keeps every date, so the missing days had to be missing from the input.
2. A date-by-date check per stage was suggested; `st_raw` already had only the 28th.
3. I guessed the date conversion and then mismatched column headers. Tests ruled out both: reading never collapses real dates to one day, and both files had identical headers.
4. Per-file date ranges showed `20261029_052117.xlsx` and `20261029_052120.xlsx`: 20,808 rows, all on the 28th, starting at midnight. So it wasn't a cut-off export either.
5. Your manual read of `Videos\...\CDR\Stringee\20261030_122403 (1).xlsx` started on 1 Oct, and the folder difference was the answer.

The file list printed at the start of every run (`Stringee files: [...]`) held the clue from the first run. Step 4 should have been step 1.

The check to run first, every time

```
for f, p in zip(st_files, st_parts):
    t = pd.to_datetime(p["Start time"], format="%m/%d/%Y %I:%M:%S %p",
errors="coerce")
    print(f"{f.parent.name}/{f.name}: {len(p):,} rows | {t.min()} ->
{t.max()}")
```

Printing the folder name too would have shown `st_daily` against the expected `Stringee` at once.

Reading `10/28/2026 09:34:25 PM` safely

```
st_raw["Start time"] = pd.to_datetime(st_raw["Start time"], format="%m/%d/%Y
%I:%M:%S %p", errors="coerce")
unread = st_raw["Start time"].isna().sum()
if unread:
    raise ValueError(f"{unread} Stringee start times could not be read")
```

- **`%m/%d/%Y`** reads month first. A guessing parser read `11/03/2026` as 11 March in testing.
- **`%I` with `%p`** is the 12-hour clock with AM/PM.
- **Real Excel dates** that only *display* like this pass through unchanged; the milliseconds in your data (`.181000`) show yours are real dates.
- **`errors="coerce"` plus the `unread` check** stops the run instead of quietly dropping rows.

Practice

- [ ] Change the file-list print so it shows each file's full folder path and modified date.

8. Small settings changes: failed dials kept, team list from a folder

Two changes made the script fit how you work: `CAN_NOT_MAKE_CALL` rows now count as dialed calls, and the team list is whichever file is newest in its folder.

Keeping `CAN_NOT_MAKE_CALL`

```
EXCLUDE_STRINGEE_END_CODES = []      # keep every end code
```

Column	Effect of keeping them
`Total_Dialed_Calls` , hourly `Calls` , `Total Calls`	Go up: each failed dial counts
Connected calls, talk time, `_GT_30`	No change: these calls are never "Answered"
`Unique_Dialed_Numbers`	A number counts once even if the attempt failed

Column	Effect of keeping them
Gaps and login hours	Barely change; a failed dial can split one long gap into two shorter ones

See how many there are with `st["End of Call code"].value_counts()`.

Reading the newest team list in a folder

```
TEAM_DIR = Path(r"C:\Users\Administrator\Videos\Office\Dialer\Team List")

team_files = sorted(f for f in TEAM_DIR.iterdir()
                    if f.suffix.lower() in (".csv", ".xlsx") and not
f.name.startswith("~$"))
team_file = max(team_files, key=lambda f: f.stat().st_mtime)      # newest
file
ref = (pd.read_csv(team_file, dtype={"Dialer Name": str}) if
team_file.suffix.lower() == ".csv"
       else read_xlsx(team_file, dtype={"Dialer Name": str}))
```

- **`f.stat().st_mtime`** is the file's last-modified time; `max` picks the newest.
- **Only one file is used.** Combining an old and a new team list would give duplicate or conflicting CRM IDs.
- **The run prints which file it chose**, and lists all of them if there's more than one.

Practice

- [ ] Print how many CRM IDs are in the chosen team list and how many are marked inactive.

9. Uno API: storing the token and fetching by date

The token is now stored as the Windows environment variable `UNO_API_TOKEN`, but the call report endpoint is still unknown. Uno's status names and call types match DeepCall (by Sarv), which offers call-report APIs. That match is from search results only, and needs confirming in the Uno panel's API section.

The error: `KeyError: '<the token>'`

`os.environ["NAME"]` looks a variable up by its **name**. The code passed the token's **value**, and no variable has that name.

```
USER_ID = "30628867"                    # not secret, so it can go in
directly
TOKEN = os.environ["UNO_API_TOKEN"]     # the variable's name, not the token
```

Create the variable once in Command Prompt or PowerShell, then fully close and reopen VS Code, because programs only read environment variables when they start:

```
setx UNO_API_TOKEN "your-token"
```

Check it with `print("UNO_API_TOKEN" in os.environ)`; it should print `True`.

Keeping the token safe

- The token was pasted into chat, so treat it as exposed: regenerate it in the Uno panel and run `setx` again with the new one.
- Never put a token in a notebook, a file you share, or a chat. Code reads it by name, so swapping tokens needs no code change.

The fetch plan (fill in from the Uno docs)

Step	Code idea
One day at a time	`while d <= END: fetch_day(d); d += timedelta(days=1)`
Page through each day	Loop `page += 1` until a page comes back short
Stop on a bad request	`r.raise_for_status()`
Be gentle with the server	`time.sleep(0.5)` between pages
Save for the dialer script	`pd.json_normalize(rows).to_csv(UNO_DIR / ...)`

API fields won't match the panel export. If it follows DeepCall's format, the agent is a separate `masterAgent` field, talk time is `talkDuration` in seconds, and status is a number where 3 means Both Answered. The Uno section of the dialer script will need a

small mapping.

Still needed

- [ ] The call report endpoint URL, parameter names and a sample response (token removed), from the Uno panel.

10. Client activity: trading days per month, not bills

Count distinct dates with at least one BILL, not BILL rows. A month can have at most 31 trading days, so the values 155, 229 and 446 in the first report meant it was still counting bills.

The corrected cell

```
bill_led = led[led['LedgerType'] == 'BILL'].copy()
bill_led = bill_led[bill_led['ClientID'].isin(check)]
bill_led['Month'] = bill_led['CreatedAt'].dt.to_period('M')
bill_led['Trade Date'] = bill_led['CreatedAt'].dt.normalize()

pivot = bill_led.pivot_table(index='ClientID', columns='Month', values='Trade
Date',
                             aggfunc='nunique', fill_value=0)
month_cols = pivot.columns
assert pivot[month_cols].max().max() <= 31, "Still counting bills, not days"

pivot['Total Trading Days'] = pivot[month_cols].sum(axis=1)
pivot['Average'] = pivot[month_cols].mean(axis=1).round(2)
pivot = pivot.reset_index()
pivot.columns = pivot.columns.astype(str)
```

Client's bills	`count` gives	`nunique` of `Trade Date` gives
5 bills on 3 Oct, 2 on 4 Oct	7	2
40 bills on one day	40	1

How it works

- **`.dt.normalize()`** drops the time, so every bill on the same day shares one date.

- **`aggfunc='nunique'`** counts different dates per client per month.
- **`assert ... <= 31`** stops the run if the numbers are impossible, instead of producing a wrong file.
- **`Total Trading Days`** adds the months; each date sits in exactly one month.

Why the wrong file appeared

The screenshot had no `Total Trading Days` column, so it came from the old pivot: either the new cell didn't replace it, or the export couldn't overwrite a file still open in Excel. Close the file, then run the whole cell.

Other fixes in that code

- `format='%Y-%m-%d %H:%M-%S'` had `-` instead of `:`. `get_final_ledger()` already returns dates, so the line can go.
- `.copy()` stops the `SettingWithCopyWarning`.
- The two date columns are now named `Last Transaction` and `Last Trade` instead of `Last Trans` and `CreatedAt`.
- `########` in Excel only means the column is too narrow for a date.

Read the numbers with care

- Months with no trades count as 0 in `Average`, including months before a client joined.
- Day boundaries are UTC; convert with `tz_localize('UTC').tz_convert('Asia/Dubai')` for Dubai days.

Practice

- [ ] Change `Average` to cover only months from each client's first trade onwards.

Error log

Today's errors were mostly about inputs and names: the wrong folder, a value used as a name, and an output from an older run.

Error or symptom	Tool	Cause	Fix	Section
`FILTER` formula rejected	Excel	`'BILL'` in single quotes	`"BILL"`; single quotes only for sheet names	1
`IndexError` with fewer than 2 files	pandas	`if len(files) > 2` let 0 or 1 file through	`!= 2`, listing the files found	2
Phone numbers like `919225131053.0`	pandas	A blank cell made the column float	`dtype=str` at read time	3
Durations show as `0.003935` in Excel	pandas	Timedeltas written as fractions of a day	Write numbers with an `[h]:mm:ss` format	3
Uno calls all in `20+`, durations 0	pandas	Old notebook glued a date to a full date-time; dropped non-text durations	Rewritten Uno section	4
Hourly talk time 1 second short	pandas	`int(mins * 60)` float rounding	Work in whole seconds	4
`Grouper for 'Date' not 1-dimensional`	pandas	`Date` was a pivot key and the counted value	Count `"Call Status"`	6
Report only had the 28th	pandas	`STRINGEE_DIR` pointed to an old folder	Point it at `Videos\...\CDR\Stringee`	7
`KeyError: '<token>'`	Python	Token value used as the variable name	`os.environ["UNO_API_TOKEN"]` after `setx`	9

Error or symptom	Tool	Cause	Fix	Section
Over 31 trading days in a month	pandas	Report from the old bill-count pivot	`nunique` of dates; `assert <= 31`	10
`########` in date columns	Excel	Column too narrow	Widen the column	10

Cheat sheet: new today

Only functions that weren't in the 29 or 30 Oct cheat sheets.

pandas

Function	What it does	Example
`pd.read_excel(path, engine="calamine")`	Much faster Excel reading	Stringee exports
`pd.ExcelWriter(path, engine="xlsxwriter")` + `set_column(i, i, w, fmt)`	Column formats when writing	`[h]:mm:ss` durations
`pd.to_timedelta(s.astype(str), errors="coerce")`	Text or Excel times to durations	`00:05:40`
`td.dt.total_seconds() / 86_400`	Duration to Excel's fraction-of-a-day	Writing durations
`df.groupby(keys).col.shift()`	Previous row's value within each group	Previous call's start
`series.clip(lower=pd.Timedelta(0))`	Floor negative values at zero	Overlapping calls
`sort_values(..., kind="mergesort")`	Stable sort: ties keep their order	Repeatable gaps
`pd.to_datetime(s, format="%m/%d/%Y %I:%M:%S %p")`	Month-first, 12-hour text dates	`10/28/2026 09:34:25 PM`

Function	What it does	Example
`pivot_table(..., aggfunc="nunique")`	Distinct values per cell	Trading days per month
`series.dt.to_period("M")`	Date to its month	`2026-10` columns
`pd.json_normalize(rows)`	List of JSON records to a table	API responses

Python

Function	What it does	Example
`Path.glob("*.xlsx")`, `Path.iterdir()`	List files in a folder	Input files
`f.stat().st_mtime`	A file's last-modified time	Newest team list
`os.environ["NAME"]`	Read an environment variable by name	`UNO_API_TOKEN`
`setx NAME "value"` (terminal)	Save a Windows environment variable	Storing the token
`assert condition, "message"`	Stop if something impossible happens	Days per month <= 31
`requests.get(url, params=..., timeout=60)`	Call an API	Uno call reports
`r.raise_for_status()`	Raise an error on a failed request	Bad token or URL
`# %%` and `# %% [markdown]`	Notebook cells in a `.py` file	`dialer_report.py`

Excel

Item	What it does	Example
`'Sheet name'!A:A`	Single quotes around a sheet name	`'31 oct 2026'!C:C`

Item	What it does	Example
`"text"`	Double quotes around text	`"BILL"`
`[h]:mm:ss` number format	Durations over 24 hours display correctly	Talk time totals

Open items

The dialer report needs its first full October run, and the Uno API is waiting on the endpoint details.

- [ ] Run `dialer_report.py` on 1 to 30 Oct and check `Calls per day` and `Hourly totals match the summary: True`.
- [ ] Tidy the settings cell: remove the duplicated `read_xlsx`, `write_xlsx`, `LEVERAGE_PER_CALL`, `LONG_CALL_SECONDS`, `ATTENDANCE_BANDS` and `KEEP_ATTENDANCE` definitions.
- [ ] Check `UNO_DIR` points to the folder that really holds the Uno CSV.
- [ ] Pick the Uno connected rule (`answered_flag` or `both_answered`) using the Status by Answered table.
- [ ] Delete the duplicate processed file (`st_daily_2026-10-28.xlsx` vs `..._to_2026-10-28.xlsx`) and the duplicate download `20261030_122403 (1).xlsx` if the original is also there.
- [ ] Regenerate the Uno token and run `setx UNO_API_TOKEN` again.
- [ ] Get the Uno call report endpoint, parameters and a sample response.
- [ ] Rerun the client activity cell and confirm the highest month is 31 or less.
- [ ] Open from earlier days: the negative balance protection `SUMIFS`, the ark files showing `59:48.4`, and the manager's decision on the 205 automatic traders.
- [ ] PG40 Day 1, exercises 1 to 5.